

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ  
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки  
09.03.01 Информатика и вычислительная техника  
направленность (профиль)  
«Технологии разработки программного обеспечения»

**Выпускная квалификационная работа**

Разработка CLI и веб-интерфейса для системы нагрузочного тестирования

Обучающегося 4 курса  
очной формы обучения  
Адаменко Семёна Сергеевича

Руководитель выпускной квалификационной  
работы:  
кандидат педагогических наук, доцент, доцент ка-  
федры информационных технологий и электрон-  
ного обучения  
Государев Илья Борисович

Санкт-Петербург  
2026

## СОДЕРЖАНИЕ

|                                                                     |    |
|---------------------------------------------------------------------|----|
| ВВЕДЕНИЕ                                                            | 3  |
| Глава 1. Аналитическая часть                                        | 6  |
| 1.1 Анализ существующих инструментов нагрузочного тестирования . .  | 6  |
| 1.2 Требования к разрабатываемому инструменту . . . . .             | 8  |
| 1.3 Выбор технологий и архитектурных решений . . . . .              | 11 |
| 1.4 Проектирование архитектуры системы . . . . .                    | 15 |
| Глава 2. Проектная часть                                            | 21 |
| 2.1 Реализация движка нагрузочного тестирования . . . . .           | 21 |
| 2.2 Реализация CLI-интерфейса . . . . .                             | 25 |
| 2.3 Реализация серверной части и веб-интерфейса . . . . .           | 28 |
| 2.4 Тестирование системы . . . . .                                  | 32 |
| 2.5 Сборка, дистрибуция и CI/CD проекта . . . . .                   | 35 |
| ЗАКЛЮЧЕНИЕ                                                          | 38 |
| Список литературы                                                   | 40 |
| ПРИЛОЖЕНИЯ                                                          | 42 |
| Приложение А. Исходный код конечного автомата тестовой сессии       | 42 |
| Приложение Б. Конфигурация GoReleaser                               | 46 |
| Приложение В. Интеграционные тесты движка нагрузочного тестирования | 48 |
| Приложение Г. Пример JSON-отчёта результатов нагрузочного теста     | 52 |

## ВВЕДЕНИЕ

В современных условиях ускоренного роста цифровых сервисов обеспечение их надёжности и производительности становится одной из ключевых задач разработки программного обеспечения. Нагрузочное тестирование позволяет выявить узкие места системы до её выхода в промышленную эксплуатацию, оценить поведение сервиса при пиковых нагрузках и проверить соответствие нефункциональным требованиям. Среди широко применяемых инструментов — Apache JMeter, Gatling, k6, Vegeta, hey. Однако большинство из них требуют тяжёлых зависимостей, ориентированы исключительно на HTTP или лишены встроенного веб-интерфейса для визуального мониторинга теста в реальном времени.

**Актуальность** работы обусловлена потребностью в лёгком инструменте нагрузочного тестирования, распространяемом в виде единого скомпилированного бинарного файла, способном работать с HTTP и gRPC сервисами, интегрироваться в конвейеры CI/CD через REST API и предоставлять наглядный веб-интерфейс для контроля хода тестирования.

**Объект исследования** — процесс нагрузочного тестирования веб-сервисов и gRPC-интерфейсов.

**Предмет исследования** — программные методы разработки CLI и веб-интерфейса инструмента нагрузочного тестирования с поддержкой HTTP и gRPC.

**Теоретическая значимость** работы заключается в систематизации подходов к проектированию высокопроизводительных CLI-инструментов на языке Go, анализе механизмов управления параллельными запросами и ограничение скорости, а также в обосновании архитектурных решений для потоковой передачи метрик через WebSocket.

**Практическая значимость** работы состоит в разработке готового к применению инструмента Tsunami, который позволяет: нагружать HTTP и gRPC сервисы с настраиваемой частотой запросов; отслеживать метрики в режиме реального времени через браузерный интерфейс; экспортировать результаты в формате JSON для последующего анализа; интегрировать тестирование в автоматизированные конвейеры через REST API. Разработанный инструмент распространяется в виде открытого

программного обеспечения для шести платформ через GitHub Releases и Homebrew, что обеспечивает его доступность для широкого круга специалистов: инженеров по качеству, DevOps-специалистов и преподавателей дисциплин по тестированию программного обеспечения. Кроме того, проект может служить учебным примером применения Go-технологий (горутины, WebSocket, gRPC) при подготовке специалистов в области разработки программного обеспечения.

**Цель** выпускной квалификационной работы — разработать CLI-инструмент и веб-интерфейс для системы нагрузочного тестирования HTTP и gRPC сервисов, обеспечивающих гибкое управление нагрузкой и визуализацию метрик в реальном времени.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести сравнительный анализ существующих инструментов нагрузочного тестирования.
2. Сформулировать функциональные и нефункциональные требования к разрабатываемому инструменту.
3. Спроектировать архитектуру системы: движок нагрузочного тестирования, CLI-слой и серверную часть.
4. Реализовать модуль нагрузочного тестирования с поддержкой протоколов HTTP и gRPC.
5. Реализовать CLI-интерфейс и веб-интерфейс системы для управления нагрузочными тестами и мониторинга метрик в реальном времени.
6. Провести тестирование разработанного инструмента и оценить его производительность.

**Результатом** бакалаврской выпускной квалификационной работы является инструмент нагрузочного тестирования Tsunami — программное средство с интерфейсом командной строки и встроенным веб-приложением, реализующее нагрузочное тестирование HTTP- и gRPC-сервисов с отображением метрик в режиме реального времени.

**Структура бакалаврской выпускной квалификационной работы.** Выпускная квалификационная работа состоит из введения, двух глав, заключения, списка литературы и приложений. В первой главе проводится анализ предметной области: рассматриваются существующие инструменты нагрузочного тестирования, обосновывается выбор технологий и проектируется архитектура системы. Во второй главе описыва-

ется практическая реализация: движок нагрузочного тестирования, CLI-интерфейс, серверная часть с поддержкой WebSocket и веб-приложение на React.

## ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

### 1.1 АНАЛИЗ СУЩЕСТВУЮЩИХ ИНСТРУМЕНТОВ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ

В условиях роста требований к надёжности веб-сервисов нагрузочное тестирование стало неотъемлемой практикой в процессе разработки программного обеспечения. Нагрузочное тестирование позволяет воспроизвести условия реальной эксплуатации, выявить узкие места производительности и оценить поведение сервиса при пиковом трафике ещё до развёртывания в промышленной среде. Без систематического проведения нагрузочных испытаний разработчики рискуют обнаружить деградацию производительности уже в условиях промышленной эксплуатации, когда стоимость устранения проблем возрастает.

Существующие инструменты нагрузочного тестирования можно разделить на несколько категорий: тяжёлые корпоративные решения с графическим интерфейсом, скриптовые инструменты на базе JVM, лёгкие CLI-утилиты и облачные платформы. Для обоснования разработки собственного инструмента был проведён сравнительный анализ наиболее распространённых открытых решений.

**Apache JMeter** — один из старейших и наиболее функциональных инструментов нагрузочного тестирования с открытым исходным кодом [1]. Распространяется в виде Java-приложения и требует наличия JVM. JMeter поддерживает широкий спектр протоколов: HTTP, HTTPS, FTP, JDBC, LDAP и другие, обладает развитым графическим интерфейсом для визуального конструирования планов тестирования. Вместе с тем инструмент отличается высокой сложностью настройки: конфигурация хранится в XML-формате, а дистрибутив занимает более 60 МБ. Потребление памяти JVM существенно ограничивает масштабируемость на машинах с ограниченными ресурсами, а интеграция в конвейеры CI/CD требует дополнительной настройки плагинов и сторонних инструментов разбора отчётов.

**Gatling** — высокопроизводительный инструмент нагрузочного тестирования, ориентированный на HTTP-протокол [2]. Написан на Scala и работает на платформе JVM, что влечёт те же зависимости среды выполнения, что и Apache JMeter. Сценарии тестирования описываются на предметно-ориентированном языке (DSL) на основе Scala, что требует от пользователя базовых знаний этого языка. Gatling генерирует

подробные HTML-отчёты с перцентилями задержки и графиками распределения, однако поддержка gRPC доступна только в коммерческой редакции Gatling Enterprise.

**k6** — современный инструмент нагрузочного тестирования с открытым исходным кодом, разработанный компанией Grafana Labs [3]. Бинарный файл написан на языке Go, однако сценарии нагрузочных тестов описываются на JavaScript, что требует знания этого языка и усложняет автоматизацию в простых сценариях. k6 поддерживает HTTP, WebSocket и gRPC, предоставляет встроенную интеграцию с Grafana Cloud для визуализации результатов. Установка предполагает использование предкомпилированных исполняемых файлов или среды Node.js при написании расширений.

**Vegeta** — минималистичный HTTP-инструмент нагрузочного тестирования, написанный на языке Go [4]. Распространяется в виде единого бинарного файла без внешних зависимостей, что делает его удобным для CI/CD-сред. Vegeta поддерживает только протокол HTTP и не имеет встроенного веб-интерфейса для мониторинга хода теста в реальном времени. Конфигурация передаётся через стандартный ввод в текстовом формате, что ограничивает сценарии использования динамически формируемыми нагрузками.

**hey** — простой HTTP-инструмент нагрузочного тестирования на Go, ориентированный на быструю проверку производительности одного маршрута [5]. Не поддерживает gRPC, лишён веб-интерфейса и расширенных метрик ошибок; результат выводится только в виде текстового отчёта по завершении теста, что не позволяет отслеживать динамику метрик в ходе нагрузочного испытания.

Результаты сравнительного анализа представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ инструментов нагрузочного тестирования

| Инструмент    | Язык / ран-тайм | Протоколы       | Интерфейс | Зависимости | Веб-интерфейс        |
|---------------|-----------------|-----------------|-----------|-------------|----------------------|
| Apache JMeter | Java (JVM)      | HTTP, gRPC, FTP | GUI, CLI  | JVM         | Сторонние плагины    |
| Gatling       | Scala (JVM)     | HTTP            | CLI, HTML | JVM         | HTML-отчёт (статика) |
| k6            | Go + JS         | HTTP, WS, gRPC  | CLI       | Node.js     | Grafana Cloud        |
| Vegeta        | Go              | HTTP            | CLI       | Нет         | Отсутствует          |
| hey           | Go              | HTTP            | CLI       | Нет         | Отсутствует          |

Анализ таблицы 1.1 позволяет сделать следующие выводы. Инструменты на базе JVM обладают широким набором функциональных возможностей, однако требуют

наличия среды выполнения Java и потребляют значительный объём оперативной памяти, что затрудняет их использование в контейнерных и CI/CD-средах. Инструменты на Go являются лёгкими и не имеют зависимостей, однако ограничены только протоколом HTTP и не предоставляют визуального мониторинга. k6 поддерживает gRPC и имеет интеграцию с Grafana, но для написания сценариев требует знания JavaScript, а встроенная визуализация доступна только через платный облачный сервис.

Таким образом, на рынке отсутствует инструмент, объединяющий лёгкость и отсутствие зависимостей исполняемого файла на Go, поддержку HTTP и gRPC в одном дистрибутиве и встроенный веб-интерфейс с потоковой передачей метрик в реальном времени. Данный пробел и определяет необходимость разработки инструмента Tsunami.

## 1.2 ТРЕБОВАНИЯ К РАЗРАБАТЫВАЕМОМУ ИНСТРУМЕНТУ

На основе проведённого анализа предметной области и выявленных недостатков существующих решений были сформулированы требования к разрабатываемому инструменту нагрузочного тестирования.

### **Функциональные требования:**

1. Поддержка нагрузочного тестирования HTTP-сервисов с настраиваемыми методом запроса, заголовками и телом.
2. Поддержка нагрузочного тестирования gRPC-сервисов с возможностью указания .proto-файла или использования server reflection.
3. Гибкое управление частотой запросов в формате N/Vunit (например, 100/1s, 50/500ms).
4. Настройка числа параллельных воркеров и размера пула HTTP-соединений.
5. Сбор и отображение метрик: общее число запросов, успехи, отказы, минимальная, средняя и максимальная задержка, перцентили p50, p90, p95, p99, пропускная способность (RPS), объём переданных данных.
6. Классификация ошибок по типам: таймаут, отказ в соединении, ошибка DNS, ошибка TLS, прочие.
7. Разбивка результатов по HTTP-кодам ответов.
8. Веб-интерфейс для запуска тестов и визуализации метрик в реальном времени.
9. REST API для управления тестами из внешних систем (CI/CD).



10. Экспорт результатов в формате JSON.
11. Встроенная поддержка graceful shutdown по сигналам SIGINT/SIGTERM.

**Нефункциональные требования:**

1. Распространение в виде единого скомпилированного бинарного файла без внешних зависимостей среды выполнения.
2. Кросс-платформенная поддержка: Linux, macOS, Windows; архитектуры amd64 и arm64.
3. Корректная работа при целевой частоте не менее 1000 RPS при 50 параллельных воркерах на современном серверном оборудовании.
4. Потокбезопасный сбор метрик без потери данных при высокой нагрузке.
5. Задержка доставки метрик через WebSocket — не более 100 мс.

Сформулированные функциональные требования определяют совокупность сценариев взаимодействия пользователей с системой, которые целесообразно формализовать в виде диаграммы прецедентов. Система Tsunami ориентирована на две категории пользователей: специалиста, работающего через интерфейс командной строки, и специалиста, управляющего тестами через браузерный веб-интерфейс. Диаграмма прецедентов, отражающая полный перечень вариантов использования обеих категорий, представлена на рисунке 1.1.

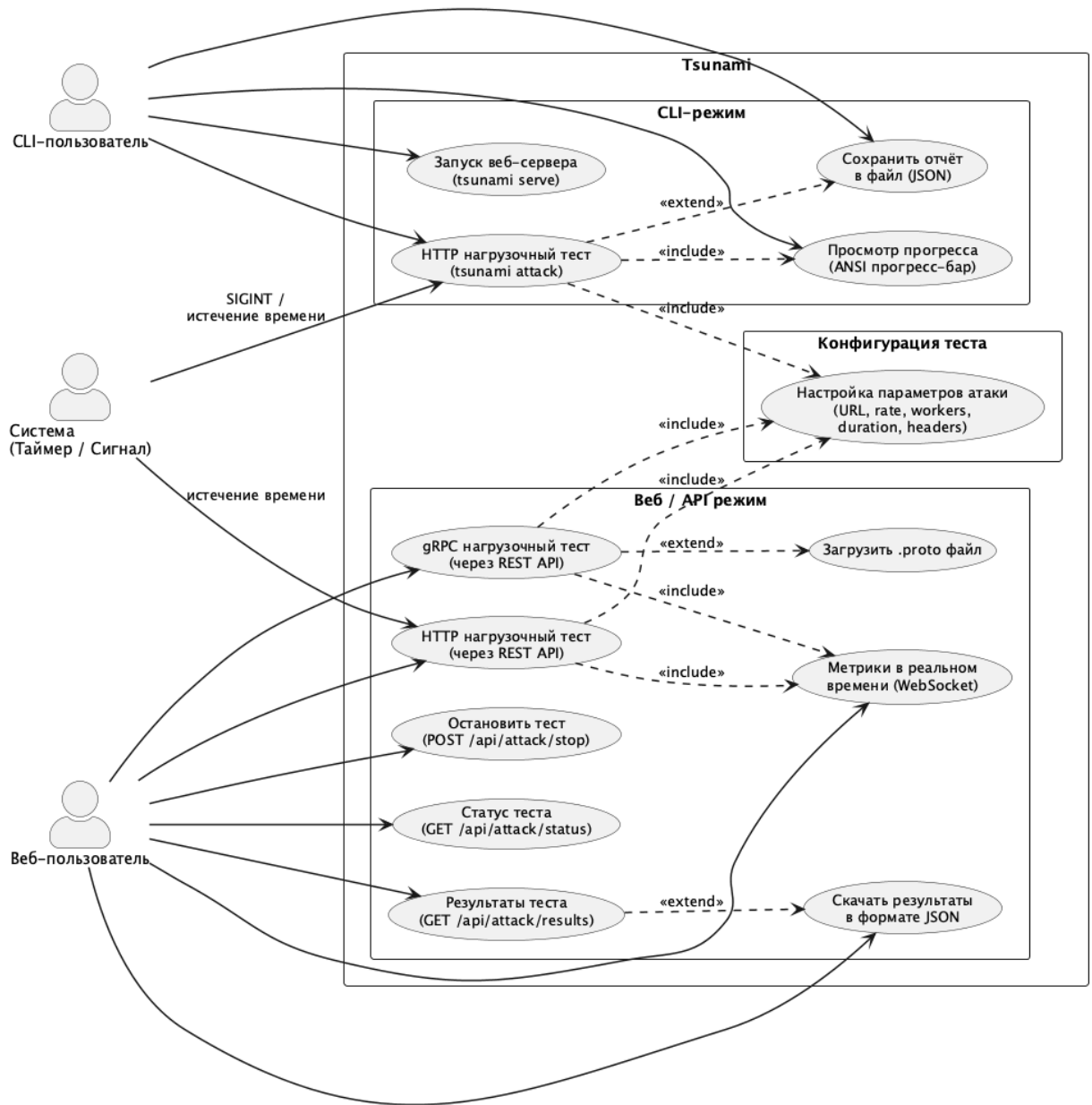


Рисунок 1.1 – Диаграмма прецедентов системы Tsunami

Как видно из рисунка 1.1, обе категории пользователей разделяют единый модуль конфигурации атаки, однако реализуют принципиально различные сценарии взаимодействия с системой. CLI-пользователь взаимодействует с системой через команды `attack`, `serve` и `grpc`, наблюдает за ходом выполнения теста в терминале и сохраняет отчёт в файл. Веб-пользователь управляет тестами через браузерный интерфейс, получает метрики в реальном времени по протоколу `WebSocket`, останавливает тест вручную и скачивает результаты. Обе категории пользователей используют общий модуль конфигурации атаки.

### 1.3 ВЫБОР ТЕХНОЛОГИЙ И АРХИТЕКТУРНЫХ РЕШЕНИЙ

Обоснованный выбор технологического стека является необходимым условием для достижения поставленных функциональных и нефункциональных требований. В данном разделе приводится сравнительный анализ альтернатив и обоснование принятых решений по каждому компоненту системы.

#### Язык программирования

Для реализации ядра инструмента рассматривались три языка: Go, Python и Rust.

**Python** обладает развитой экосистемой для сетевого программирования (asyncio, aiohttp), однако глобальная блокировка интерпретатора (GIL) ограничивает параллельное выполнение CPU-нагрузки в нескольких потоках. Кроме того, Python-приложение требует наличия интерпретатора на целевой машине, что противоречит нефункциональному требованию о поставке единым бинарным файлом.

**Rust** обеспечивает наивысшую производительность и гарантии безопасности памяти на уровне системы типов, однако обладает высоким порогом вхождения и значительно увеличивает время разработки по сравнению с Go.

**Go** был выбран как оптимальный язык по совокупности характеристик [6]:

- горутины — легковесные потоки исполнения с начальным размером стека 2 КБ, позволяющие эффективно реализовать пул из сотен параллельных воркеров;
- встроенные примитивы конкурентности — каналы (chan), пакет sync, контексты (context.Context) — обеспечивают безопасную координацию горутин;
- статическая компиляция в единый бинарный файл без зависимостей среды выполнения при установке флага CGO\_ENABLED=0;
- богатая стандартная библиотека: net/http с пулом соединений, crypto/tls, encoding/json;
- простая кросс-компиляция под любую целевую платформу через переменные окружения GOOS и GOARCH.

## CLI-фреймворк

Для организации интерфейса командной строки сравнивались библиотеки `spf13/cobra` и `urfave/cli`.

`urfave/cli` предоставляет простой API, достаточный для однокомандных утилит, однако слабо поддерживает вложенные субкоманды и не имеет встроенной генерации документации.

`cobra` — стандарт де-факто для CLI-инструментов в экосистеме Go, используется в таких проектах, как Docker, Kubernetes, Helm и GitHub CLI [7]. Библиотека предоставляет гибкую систему субкоманд, автоматическую генерацию справочного текста, поддержку персистентных флагов и хуки `PersistentPreRun` для валидации входных параметров. По результатам сравнения выбрана библиотека `cobra`.

## Ограничение частоты запросов

Для точного управления скоростью отправки запросов рассматривались три подхода: `time.Ticker` из стандартной библиотеки, `golang.org/x/time/rate` (алгоритм `token bucket`) и `go.uber.org/ratelimit` (алгоритм `leaky bucket`).

`time.Ticker` генерирует тики с фиксированным интервалом, однако при задержке обработки тики накапливаются, что приводит к пиковым всплескам запросов вместо равномерного потока.

`golang.org/x/time/rate` реализует `token bucket`: токены накапливаются при простое и могут быть истрачены за один такт в режиме `burst`, что нежелательно при воспроизводимом нагрузочном тестировании.

`go.uber.org/ratelimit` реализует алгоритм `leaky bucket` [8]: метод `Take()` блокирует вызывающую горутину до момента, когда следующий токен может быть выдан, обеспечивая строго равномерное распределение запросов во времени. Именно такое поведение критично для нагрузочного тестирования, где требуется воспроизводимая постоянная нагрузка с заданным RPS. Выбрана библиотека `go.uber.org/ratelimit`.

## WebSocket

Для реализации потоковой передачи метрик в реальном времени сравнивались библиотеки `gorilla/websocket` и `coder/websocket`.

`coder/websocket` [9] — библиотека с контекстно-ориентированным API, однако с меньшей зрелостью и значительно меньшим сообществом пользователей.

gorilla/websocket — наиболее широко применяемая Go-реализация протокола WebSocket [10], используется в тысячах производственных проектов. Библиотека обеспечивает полное соответствие RFC 6455, поддержку компрессии и гибкую настройку параметров соединения. По результатам сравнения выбрана gorilla/websocket.

## Поддержка gRPC

Поддержка gRPC реализована на основе официального Go-пакета [google.golang.org/grpc](https://google.golang.org/grpc) [11] — единственного зрелого gRPC-фреймворка для Go, поддерживаемого командой Google. Для динамической компиляции .proto-файлов без использования внешнего компилятора protoc применяется библиотека [bufbuild/protocompile](https://github.com/bufbuild/protocompile), предоставляющая программный интерфейс к компилятору Protocol Buffers. Это позволяет пользователям передавать .proto-файлы без предварительной генерации кода.

## Фронтенд

Для разработки веб-интерфейса рассматривались три фреймворка: Angular, Vue.js и React.

**Angular** — полнофункциональный фреймворк с встроенными средствами управления состоянием и строгой структурой проекта. Для одностраничной панели мониторинга Angular является избыточным: высокий порог вхождения и большой объём итоговой сборки не соответствуют требованию о встраивании фронтенда непосредственно в исполняемый файл.

**Vue.js** — прогрессивный фреймворк с простым API, однако экосистема библиотек для визуализации данных в реальном времени уступает React-экосистеме по зрелости и разнообразию компонентов.

**React** в связке с TypeScript был выбран как оптимальное решение [12]: компонентный подход обеспечивает модульность, богатая экосистема предоставляет готовые решения для всех задач, а строгая типизация TypeScript снижает вероятность ошибок во время выполнения. Для визуализации метрик выбрана библиотека **Recharts** — декларативная SVG-библиотека графиков. Стилизация реализована с помощью **Tailwind CSS**, обеспечивающего подход к оформлению без необходимости писать отдельные CSS-файлы.

Встраивание скомпилированного React SPA в исполняемый файл выполнено с помощью директивы `//go:embed`, что позволяет распространять единый исполняемый файл, содержащий как серверную, так и клиентскую часть приложения [6].

Итоговый технологический стек представлен в таблице 1.2.

Таблица 1.2 – Технологический стек инструмента Tsunami

| Компонент        | Технология             | Обоснование выбора                                       |
|------------------|------------------------|----------------------------------------------------------|
| Язык бэкенда     | Go                     | Горутины, статический исполняемый файл, кросс-компиляция |
| CLI-фреймворк    | Cobra                  | Стандарт де-факто; субкоманды, хуки, генерация справки   |
| Rate limiting    | uber/ratelimit         | Leaky bucket; строго равномерный поток запросов          |
| WebSocket        | gorilla/websocket      | RFC 6455, зрелая библиотека                              |
| gRPC             | google.golang.org/grpc | Официальный Go gRPC-фреймворк от Google                  |
| Компилятор proto | bufbuild/protoc        | Динамическая компиляция .proto без protoc                |
| UI-фреймворк     | React + TypeScript     | Компонентность, экосистема, типобезопасность             |
| Графики          | Recharts               | Декларативные SVG-графики, интеграция с React            |
| CSS-фреймворк    | Tailwind CSS           | Утилитарный подход, малый итоговый объём сборки          |
| Встраивание SPA  | go:embed               | Единый исполняемый файл без внешних статических активов  |
| Сборщик          | Vite                   | Быстрая сборка, нативная поддержка TypeScript            |

## 1.4 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ

На основе сформулированных требований и выбранного технологического стека была спроектирована трёхслойная архитектура инструмента Tsunami, представленная на рисунке 1.2.

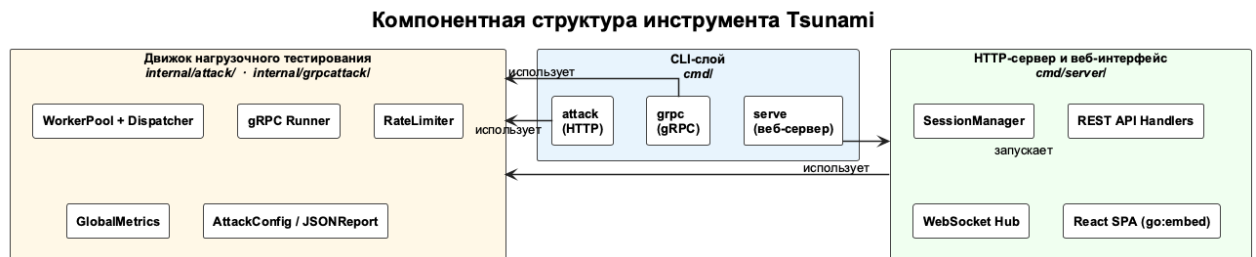


Рисунок 1.2 – Компонентная структура инструмента Tsunami

### Компонентная структура

Система состоит из трёх основных слоёв.

**CLI-слой** реализован на основе фреймворка Cobra и включает три субкоманды: attack (HTTP-тестирование), grpc (gRPC-тестирование) и serve (запуск веб-сервера). Слой отвечает за парсинг флагов командной строки, валидацию входных параметров и вывод результатов в терминал или файл.

**Движок нагрузочного тестирования** является ядром системы. Содержит конфигурационные структуры (AttackConfig), пул воркеров, диспетчер задач с ограничением скорости, потокобезопасный сборщик метрик (GlobalMetrics) и генератор JSON-отчётов.

**HTTP-сервер с веб-интерфейсом** предоставляет REST API для управления тестами, WebSocket-хаб для стриминга метрик и встроенное React-приложение.

Принципиально важным архитектурным решением является то, что CLI-слой и HTTP-сервер используют один и тот же движок нагрузочного тестирования. Это исключает дублирование бизнес-логики и гарантирует идентичное поведение системы вне зависимости от способа управления — через терминал или браузер.

### Паттерн «пул воркеров»

Паттерн «пул воркеров» позволяет управлять параллельным выполнением HTTP-запросов при фиксированном числе горутин, предотвращая неконтролируемый рост потребления памяти при высокой нагрузке. В системе Tsunami данный паттерн ре-

ализован на основе буферизованных каналов языка Go и образует трёхкомпонентную конвейерную схему: диспетчер с ограничителем скорости, пул горутин-воркеров и горутина-коллектор. Взаимодействие этих компонентов представлено на рисунке 1.3.

Как показано на рисунке 1.3, диспетчер формирует равномерный поток заданий, воркеры выполняют запросы параллельно, а коллектор агрегирует результаты без блокировки воркеров. Диспетчер задач в цикле вызывает метод `ratelimiter.Take()`, который блокирует выполнение до наступления следующего разрешённого момента согласно алгоритму `leaky bucket`. После получения токена диспетчер помещает задание в буферизованный канал `jobs`.

$N$  горутин-воркеров одновременно ожидают заданий из канала `jobs`. Получив задание, каждый воркер выполняет HTTP- или gRPC-запрос к целевому сервису, фиксирует время ответа и помещает объект `RequestResult` в канал `results`. Структура `RequestResult` содержит код ответа, задержку, признак успеха, тип ошибки и объём переданных данных.

Горутина-коллектор последовательно считывает результаты из канала `results` и накапливает их в структуре `GlobalMetrics`, защищённой мьютексом. Такое разделение ответственности позволяет воркерам не блокироваться на операциях агрегации метрик и достигать высокой пропускной способности.



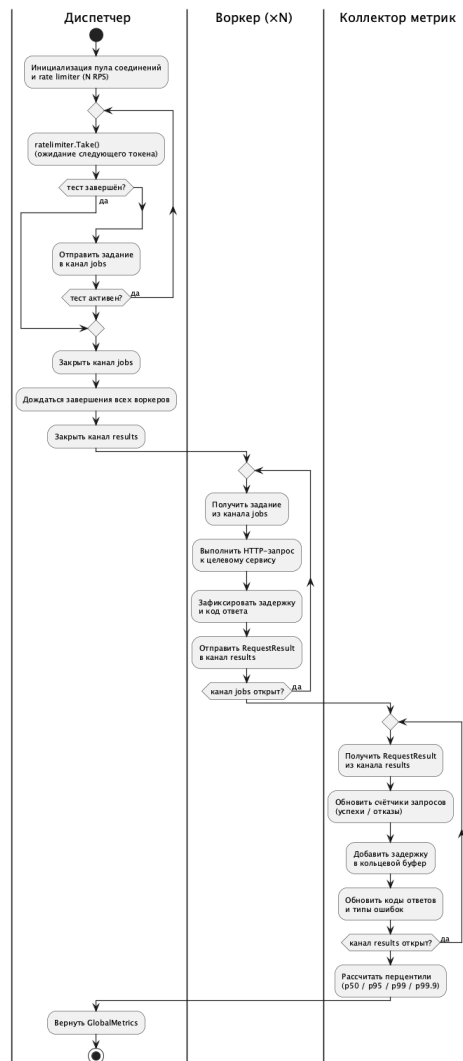


Рисунок 1.3 – Схема взаимодействия компонентов движка нагрузочного тестирования

### Структура GlobalMetrics и расчёт перцентилей

Выбор перцентилей в качестве ключевых метрик задержки обусловлен статистическими свойствами распределений, характерных для времени ответа сетевых сервисов. Среднеарифметическое значение является несмещённой оценкой математического ожидания лишь при нормальном распределении, однако распределения задержек, как правило, имеют **правостороннее асимметричное распределение** (*right-skewed distribution*): основная масса запросов обрабатывается быстро, однако редкие выбросы — вызванные сборкой мусора, перегрузкой сети или дисковыми операциями — существенно завышают среднее значение, делая его ненадёжным показателем производительности. Перцентили не зависят от величины выбросов и позволяют отдельно оценить типичное время отклика и задержку «хвоста» распределения.

**$p$ -й перцентиль** — значение выборки, не превышаемое не менее чем  $p\%$  её элементов. Для вычисления перцентиля выборка объёмом  $n$  сортируется по возрастанию, после чего определяется позиция нужного значения:

$$k = \left\lceil \frac{p}{100} \cdot n \right\rceil, \quad (1.1)$$

где  $k$  — номер элемента в отсортированном массиве (нумерация с 1),  $\lceil \cdot \rceil$  — округление вверх.

Практически значимый набор уровней  $p50$ ,  $p90$ ,  $p95$  и  $p99$  обусловлен особенностями работы распределённых систем. Уровень  $p50$  (медиана) описывает «типичный» запрос и устойчив к выбросам. Уровень  $p90$  характеризует поведение большинства запросов и служит индикатором деградации производительности на начальных стадиях перегрузки. Уровни  $p95$  и  $p99$  характеризуют **хвостовую задержку** (*tail latency*): при последовательном обращении к  $k$  независимым микросервисам вероятность того, что хотя бы один из них окажется в «хвосте»  $p99$ , равна  $1 - 0,99^k$ , что при  $k = 10$  составляет уже  $\approx 9,6\%$  всех запросов, что критично для соглашений об уровне обслуживания (SLA) [13, с. 187].

Структура GlobalMetrics обеспечивает потокобезопасный сбор всех метрик теста. Для хранения значений задержки используется кольцевой буфер (Latencies [.]time.Duration) ёмкостью 100 000 элементов. При каждом добавлении нового значения индекс сдвигается по модулю размера буфера, что гарантирует ограниченное потребление оперативной памяти вне зависимости от длительности теста и числа запросов.

Перцентили задержки ( $p50$ ,  $p90$ ,  $p95$ ,  $p99$ ) вычисляются по завершении теста функцией CalculateAllPercentiles: буфер сортируется однократно, после чего значения перцентилей считываются из соответствующих позиций отсортированного массива. Однократная сортировка вместо четырёх независимых вызовов снижает вычислительную сложность с  $O(4 \cdot n \log n)$  до  $O(n \log n)$ .

Для построения временного графика задержки используется отдельный буфер LatencyHistory ёмкостью 1 000 точек: значение задержки записывается каждые 10 запросов, что обеспечивает равномерное покрытие всего интервала теста вне зависимости от его продолжительности.

## Конечный автомат TestSession

Управление жизненным циклом теста в серверном режиме требует чётких допустимых состояний и переходов между ними, чтобы исключить некорректные операции — такие как повторный запуск активного теста или остановка уже завершённого испытания. Для решения этой задачи в системе применяется конечный автомат, инкапсулированный в объекте `TestSession` и охватывающий полный жизненный цикл нагрузочного теста. Диаграмма состояний данного автомата, включающая пять состояний и переходы между ними, представлена на рисунке 1.4.

Как видно из рисунка 1.4, автомат инициализируется в состоянии **Idle** и допускает строго детерминированные переходы: каждое состояние имеет не более одного пути к терминальным состояниям **Completed**, **Stopped** и **Error**. В состоянии **Idle** сессия создана, но тест ещё не запущен. Переход в состояние **Running** происходит при вызове метода `session.Start()`. Из состояния **Running** возможны три перехода: в **Completed** — по истечении заданной длительности теста; в **Stopped** — при явной остановке пользователем через REST API; в **Error** — при возникновении критической ошибки в движке атаки.

Остановка теста реализована через канал `StopCh` типа `chan struct`. Закрывание канала является неблокирующим сигналом для всех горутин, которые отслеживают его состояние через конструкцию `select`.

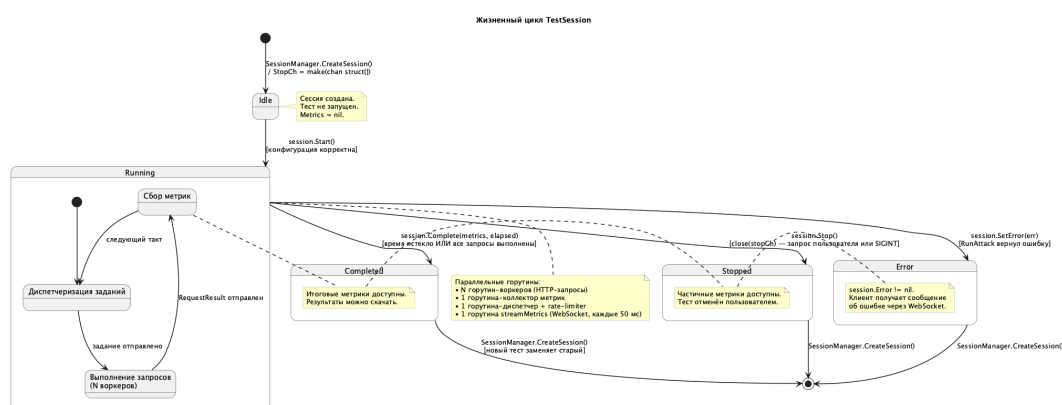


Рисунок 1.4 – Диаграмма состояний объекта `TestSession`

## WebSocket-хаб и стриминг метрик

Для трансляции метрик подключённым браузерным клиентам применяется паттерн «хаб» (Hub). Хаб поддерживает три канала: `register`, `unregister` и `broadcast`.

Горутинa `Hub.Run()` в цикле обрабатывает эти каналы: регистрирует новых клиентов, удаляет отключившихся и рассылает сообщения всем активным соединениям.

Отдельная горутинa `streamMetrics` с интервалом 50 мс считывает текущее состояние `GlobalMetrics`, формирует объект `MetricsPayload` и передаёт его в канал `broadcast` хаба. Каждый клиент имеет собственный буферизованный канал `send` ёмкостью 256 сообщений и отдельные горутини `readPump/writePump`, что предотвращает блокировку хаба при наличии медленных или зависших клиентов.

## REST API сервера

HTTP-сервер предоставляет следующие маршруты (таблица 1.3):

Таблица 1.3 – Эндпоинты REST API инструмента Tsunami

| Метод | Путь                                      | Описание                             |
|-------|-------------------------------------------|--------------------------------------|
| POST  | <code>/api/attack/start</code>            | Запуск нагрузочного теста            |
| POST  | <code>/api/attack/stop</code>             | Остановка текущего теста             |
| GET   | <code>/api/attack/status</code>           | Текущий статус и метрики теста       |
| GET   | <code>/api/attack/results</code>          | Итоговые результаты после завершения |
| GET   | <code>/api/attack/results/download</code> | Скачать результаты в формате JSON    |
| POST  | <code>/api/proto/upload</code>            | Загрузить .proto-файл для gRPC       |
| WS    | <code>/ws/metrics</code>                  | WebSocket: стриминг метрик (50 мс)   |
| GET   | <code>/health</code>                      | Проверка работоспособности сервера   |

Маршрутизация организована таким образом, что все запросы к путям `/api/`, `/ws/` и `/health` обрабатываются API-мультиплексором, а все остальные пути передаются обработчику SPA (`SPAHandler`), который возвращает `index.html` для поддержки клиентской маршрутизации React. К серверу применяется `CORS-middleware`, разрешающий кросс-доменные запросы в режиме разработки.

Таким образом, спроектированная архитектура обеспечивает чёткое разделение ответственности между слоями: CLI и веб-сервер используют единый движок нагрузочного тестирования, что исключает дублирование бизнес-логики. Применение паттернов пула воркеров, конечного автомата и WebSocket-хаба позволяет достичь высокой производительности и удобства мониторинга в реальном времени. Принятые в данной главе архитектурные решения реализуются в главе 2.

## ГЛАВА 2. ПРОЕКТНАЯ ЧАСТЬ

### 2.1 РЕАЛИЗАЦИЯ ДВИЖКА НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ

Центральным компонентом системы Tsunami является **движок нагрузочного тестирования** — модуль, отвечающий за генерацию HTTP и gRPC нагрузки с заданной интенсивностью запросов, измерение задержки и сбор агрегированных метрик. Движок предоставляет две публичные точки входа: функцию `RunAttack` для автономного запуска и `RunAttackWithMetrics` для режима реального времени, при котором потребитель передаёт заранее созданный объект метрик и может наблюдать за ними непосредственно в ходе теста. Разделение на два варианта позволяет использовать одно ядро как в CLI-режиме с живым прогрессом, так и в серверном режиме с WebSocket-трансляцией метрик. Подобная архитектура соответствует принципу единственной ответственности: движок не знает о способе представления результатов и не зависит от транспортного уровня сервера.

#### Архитектура воркер-пула

Фундаментом движка служит **воркер-пул** — классическая модель параллельного выполнения задач на основе каналов языка Go [6]. При запуске теста создаётся фиксированное число горутин-воркеров (`cfg.Workers`, по умолчанию 50), каждая из которых ожидает задания из канала `jobs`. Буфер входящего канала рассчитывается по формуле  $\max(\text{Workers} \times 2, 100)$ , что предотвращает простой диспетчера при кратковременном замедлении воркеров. Результаты каждого запроса помещаются в канал `results` ёмкостью  $\text{Workers} \times 2$ , который параллельно обрабатывает горутина-коллектор, накапливающая статистику в объекте `GlobalMetrics`.

Управление скоростью генерации нагрузки возложено на **rate limiter** библиотеки `go.uber.org/ratelimit` [8], реализующей алгоритм `leaky bucket`. Вызов `rateLimiter.Take()` в горутине блокирует её ровно на время, необходимое для соблюдения целевого RPS, не допуская накопления задержанных токенов и выброса пакетных всплесков. Диспетчер непрерывно отправляет маркеры в канал `jobs`; при получении сигнала остановки через канал `stopCh` он завершает работу, после чего закрытие канала `jobs` каскадно останавливает все воркеры. Для защиты от двойного закрытия канала применяется

`sync.Once`, что исключает паническое завершение при одновременном получении внешнего сигнала и истечении таймера длительности теста.

Листинг 2.1: Инициализация воркер-пула и rate limiter в `RunAttackWithMetrics`

```

1 jobsBufferSize := max(cfg.Workers*2, 100)
2 jobs          := make(chan struct{}, jobsBufferSize)
3 results       := make(chan RequestResult, cfg.Workers*2)
4 var wg sync.WaitGroup
5
6 for i := uint(1); i <= cfg.Workers; i++ {
7     wg.Add(1)
8     go worker(client, cfg, jobs, results, &wg)
9 }
10 rateLimiter := ratelimit.New(cfg.RPS)

```

## HTTP-клиент и классификация ошибок

Каждый воркер использует общий экземпляр `*http.Client`, созданный функцией `createHTTPClient`. Транспортный уровень конфигурируется с параметрами `MaxIdleConns` и `MaxConnsPerHost`, установленными равными `cfg.Connections` (по умолчанию 100), что обеспечивает эффективное повторное использование TCP-соединений через механизм **Keep-Alive**. Задержка измеряется от момента вызова `client.Do(req)` до завершения чтения тела ответа функцией `io.ReadAll`, что даёт корректное время отклика, включающее передачу данных.

Для точной диагностики сбоев каждая ошибка классифицируется функцией `classifyError`, которая относит её к одному из пяти типов: `Timeout`, `ConnectionRefused`, `DNS`, `TLS` или `Other`. Классификация производится по типу сетевой ошибки (`net.Error.Timeout()`) и по подстрокам сообщения об ошибке («connection refused», «no such host», «tls», «certificate»). Запрос считается успешным при `statusCode < 400`; ответы с кодами 4xx и 5xx регистрируются как отказы без категории сетевой ошибки, поскольку соединение было установлено корректно. Подобная детализация позволяет пользователю оперативно различить инфраструктурные проблемы (отказ DNS, истёкший TLS-сертификат) от прикладных ошибок сервера.

## Сбор метрик и вычисление перцентилей

Все метрики агрегируются в структуре **GlobalMetrics**. Структура встраивает `sync.Mutex`, что обеспечивает атомарное обновление счётчиков из горютины-коллектора без гонок данных. Для вычисления перцентилей задержки используется **кольцевой буфер** ёмкостью `MaxLatencySamples = 100 000` замеров; при размере `time.Duration` в 8 байт это соответствует приблизительно 800 КБ памяти независимо от длительности теста. По достижении предела новые значения замещают старые по индексу `latencyIdx`, инкрементируемому по модулю ёмкости буфера; таким образом обеспечивается ограниченное потребление памяти при неограниченном числе запросов.

Перцентили P50, P90, P95 и P99 вычисляются функцией `CalculateAllPercentiles`. Для экономии ресурсов выполняется однократная сортировка среза (функция `slices.Sort` с асимптотикой  $O(n \log n)$ ), после чего каждый перцентиль определяется по индексу  $\max(\lceil p/100 \cdot N \rceil - 1, 0)$ .

Листинг 2.2: Функция `CalculateAllPercentiles` и вспомогательная `percentileIndex`

```

1 func CalculateAllPercentiles(latencies []time.Duration) (
2     p50, p90, p95, p99 time.Duration) {
3     if len(latencies) == 0 { return }
4     sorted := make([]time.Duration, len(latencies))
5     copy(sorted, latencies)
6     slices.Sort(sorted)
7     p50 = sorted[percentileIndex(sorted, 50)]
8     p90 = sorted[percentileIndex(sorted, 90)]
9     p95 = sorted[percentileIndex(sorted, 95)]
10    p99 = sorted[percentileIndex(sorted, 99)]
11    return
12 }
13 func percentileIndex(sorted []time.Duration, p int) int {
14     return max(int(math.Ceil(
15         float64(p)/100.0*float64(len(sorted)))), -1, 0)
16 }
```

Назначение каждой метрики задержки представлено в таблице 2.1.

Таблица 2.1 – Метрики задержки, собираемые движком Tsunami

| Метрика         | Обозн. | Формула индекса                   | Интерпретация                                             |
|-----------------|--------|-----------------------------------|-----------------------------------------------------------|
| 50-й перцентиль | P50    | $\lceil 0,50 \times N \rceil - 1$ | Медианная задержка; типична для большинства запросов      |
| 90-й перцентиль | P90    | $\lceil 0,90 \times N \rceil - 1$ | Задержка при умеренной нагрузке                           |
| 95-й перцентиль | P95    | $\lceil 0,95 \times N \rceil - 1$ | Граница комфортного SLA для большинства сервисов          |
| 99-й перцентиль | P99    | $\lceil 0,99 \times N \rceil - 1$ | Хвостовая задержка; критична для систем реального времени |

### Поддержка протокола gRPC

Помимо HTTP, система поддерживает нагрузочное тестирование по протоколу **gRPC** [11], реализованное в модуле gRPC-атак. Вместо единственного `http.Client` gRPC-воркеры разделяют пул из `cfg.Connections` каналов (по умолчанию 4); каждый воркер выбирает канал по формуле `conns[i % poolSize]`, что обеспечивает равномерное распределение нагрузки между соединениями без дополнительной синхронизации. Разрешение полного имени метода осуществляется функцией `ResolveMethod` двумя способами: из предоставленного `.proto`-файла с помощью компилятора `protoc` или через `server reflection` в режиме реального времени. Коды ошибок gRPC отображаются на те же пять типов: `DeadlineExceeded` преобразуется в `Timeout`, `Unavailable` — в `ConnectionRefused` и т. д., что позволяет использовать единую систему формирования отчётов для обоих протоколов.

Сравнение конфигурационных параметров HTTP и gRPC движков приведено в таблице 2.2.

Таблица 2.2 – Сравнение конфигурации HTTP и gRPC движков Tsunami

| Параметр            | HTTP-движок                              | gRPC-движок                                         |
|---------------------|------------------------------------------|-----------------------------------------------------|
| Воркеры (умолч.)    | 50                                       | 50                                                  |
| Соединения (умолч.) | 100                                      | 4                                                   |
| Пул соединений      | <code>http.Transport (Keep-Alive)</code> | Пул <code>grpc.ClientConn</code>                    |
| Rate limiter        | <code>go.uber.org/ratelimit</code>       | <code>go.uber.org/ratelimit</code>                  |
| Буфер jobs          | $\max(Workers \times 2, 100)$            | $\max(Workers \times 2, 100)$                       |
| Разрешение метода   | —                                        | <code>.proto</code> -файл / <code>reflection</code> |
| Критерий успеха     | <code>statusCode &lt; 400</code>         | <code>grpc.Code == OK</code>                        |

Статическая структура модуля нагрузочного тестирования и взаимосвязи между его ключевыми сущностями структурированы в виде диаграммы классов. На ней отражены основные структуры данных — `AttackConfig`, `GlobalMetrics` и `RequestResult` —



а также их зависимости от функций движка и транспортного уровня. Диаграмма классов модуля нагрузочного тестирования представлена на рисунке 2.1.

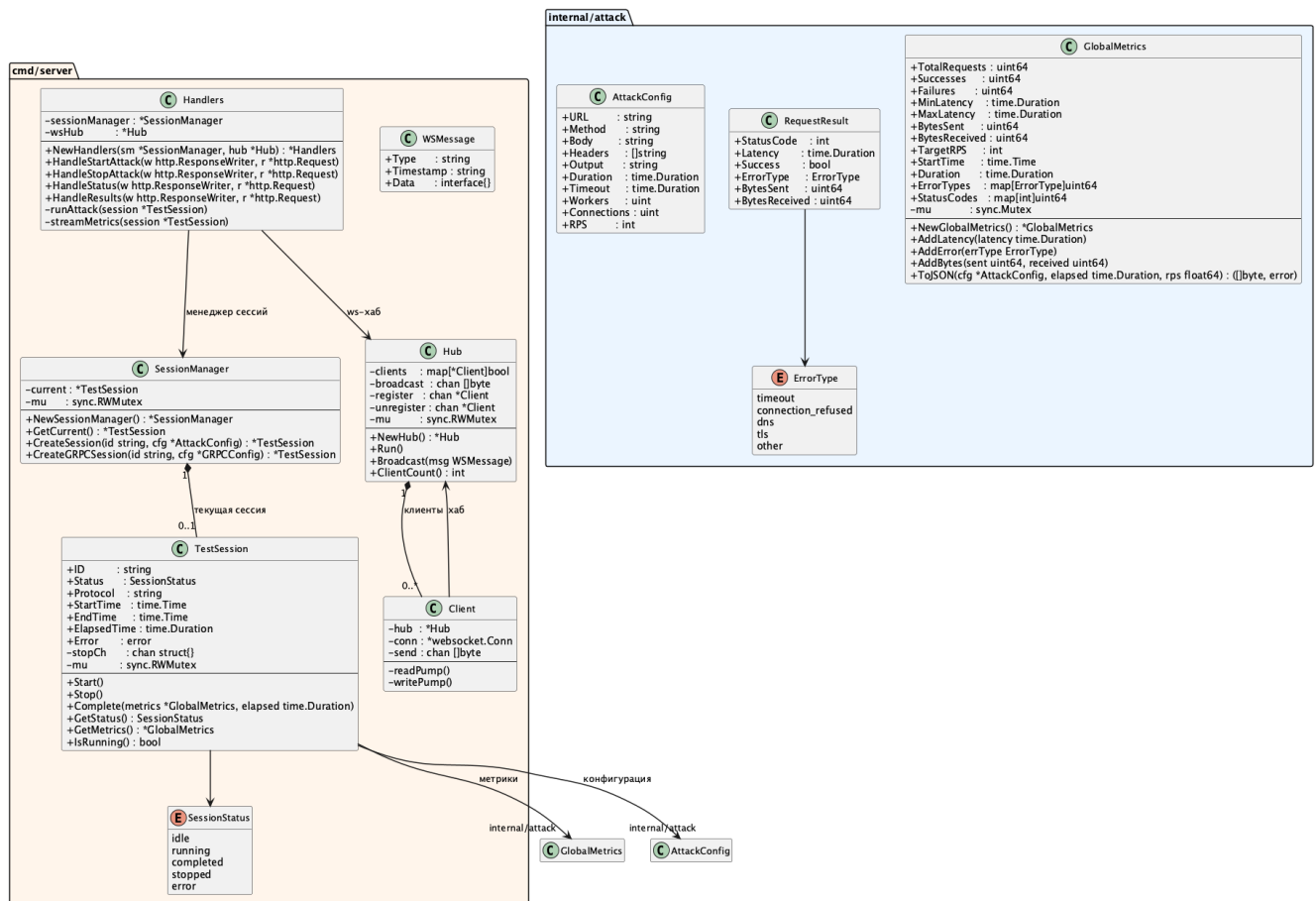


Рисунок 2.1 – Диаграмма классов модуля нагрузочного тестирования

Из рисунка 2.1 следует, что ключевые структуры чётко разграничены по ответственности: AttackConfig хранит параметры конфигурации, GlobalMetrics агрегирует статистику, а RequestResult передаёт данные от каждого воркера к горутине-коллектору. Таким образом, движок нагрузочного тестирования Tsunami обеспечивает точное соблюдение целевого RPS за счёт leaky-bucket ограничителя скорости, собирает полный набор перцентильных метрик задержки в ограниченном по памяти кольцевом буфере и поддерживает два транспортных протокола с единой системой сбора и классификации ошибок.

## 2.2 РЕАЛИЗАЦИЯ CLI-ИНТЕРФЕЙСА

Командный интерфейс является основным способом взаимодействия конечного пользователя с системой Tsunami при автономном запуске нагрузочных тестов. В качестве фреймворка для построения CLI выбрана библиотека Cobra [7] — де-факто

стандарт разработки CLI-инструментов на языке Go, применяемый в таких проектах, как Docker и Kubernetes. Cobra обеспечивает декларативное описание команд, автоматическую генерацию справочной документации и разграничение флагов на глобальные и локальные, что позволило структурировать интерфейс инструмента в виде иерархии трёх подкоманд с минимальным дублированием кода. Корневая команда `tsunami` предоставляет единственный глобальный флаг `-cpus` для явного задания числа используемых процессорных ядер (`runtime.GOMAXPROCS`).

### Структура команд и флаги

Каждая из трёх подкоманд специализирована на конкретном сценарии использования; их назначение и ключевые флаги приведены в таблице 2.3. Архитектурное разделение функциональности на отдельные команды соответствует принципу единственной ответственности и облегчает расширение инструмента новыми транспортными протоколами без модификации существующих обработчиков.

Таблица 2.3 – Подкоманды CLI-интерфейса Tsunami

| Команда | Назначение                         | Ключевые флаги                                                                                                                                                                                                                         |
|---------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| attack  | HTTP нагрузочное тестирование      | <code>-url</code> , <code>-rate</code> , <code>-duration</code> , <code>-workers</code> , <code>-connections</code> , <code>-timeout</code> , <code>-method</code> , <code>-body</code> , <code>-headers</code> , <code>-output</code> |
| serve   | HTTP-сервер с REST API и WebSocket | <code>-port</code> (по умолчанию 8080)                                                                                                                                                                                                 |
| grpc    | gRPC нагрузочное тестирование      | <code>-target</code> , <code>-service</code> , <code>-method</code> , <code>-data</code> , <code>-proto</code> , <code>-insecure</code>                                                                                                |

### Формат задания интенсивности нагрузки

Ключевым параметром команды `attack` является флаг `-rate`, задающий интенсивность нагрузки в формате `N/Vunit`, где `N` — число запросов, `V` — числовое значение временного интервала, `unit` — единица времени (`ms`, `s`, `m`, `h`). Данный формат позволяет задавать как целочисленный RPS (`100/1s`), так и субсекундные интервалы (`50/500ms`), не прибегая к вещественным числам, что исключает ошибки округления при вводе. Разбор строки выполняется функцией `ParseRateToRPS` посредством регулярного выражения; значения ниже 1 RPS округляются вверх до 1 во избежание деления на ноль в `rate limiter`.

Листинг 2.3: Регулярное выражение для разбора формата `rate`

```
1 var RateRegex = regexp.MustCompile(  
2     `^(\d+)/(\d+)(ms|s|m|h)$`)
```

### Предварительная валидация параметров

Перед запуском теста хук `PersistentPreRunE` каждой подкоманды вызывает набор проверок, что обеспечивает информативные сообщения об ошибках ещё до аллоцирования ресурсов движком. Валидируются следующие условия: URL обязан содержать схему `http://` или `https://`; строка `rate` должна соответствовать регулярному выражению `RateRegex`; количество воркеров и соединений должно быть больше нуля; тайм-аут запроса должен быть положительным. Такая схема «fail fast» соответствует идиомам языка Go [6] и избавляет пользователя от необходимости ожидать начала теста, чтобы обнаружить опечатку в параметрах. Дополнительно проверяется, что значение `-workers` не превышает допустимого максимума, заданного флагом `-max-workers`, что предотвращает случайное истощение ресурсов тестирующей машины.

### Отображение живого прогресса

В процессе выполнения HTTP-теста функция `liveProgress` запускает отдельную горутину с тикером 100 мс, которая каждые 100 миллисекунд считывает текущее состояние объекта `GlobalMetrics` и перезаписывает строку терминала через символ возврата каретки `\r`. Вывод включает анимированный спиннер, прогресс-бар для теста с конечной длительностью, текущий и целевой RPS, а также счётчики успешных и неуспешных запросов. Цвет индикатора RPS динамически изменяется в зависимости от соотношения фактического и целевого значений: зелёный — при достижении цели, жёлтый — при отставании более чем на 10%, красный — при отставании более чем на 30%. Использование ANSI escape-кодов и тикера с периодом 100 мс обеспечивает плавное обновление без мерцания и без существенного влияния на производительность самого движка.

### Корректное завершение работы

Устойчивость CLI к прерываниям обеспечивается механизмом **graceful shutdown**, реализованным в CLI-команде `attack`. После инициализации теста функция `signal.Notify` регистрирует обработчик сигналов `SIGINT` и `SIGTERM`; при

получении любого из них закрывается канал `stopCh`, что через горутины диспетчера и воркеров ведёт к плавному завершению всех «в полёте» запросов. Функция `wg.Wait()` удерживает управление вплоть до завершения последнего воркера, гарантируя, что итоговый отчёт сформируется на полном наборе результатов. Аналогичная схема завершения применяется в команде `serve`: сигналы ОС инициируют вызов `server.Shutdown` с тайм-аутом 10 секунд, после чего сервер перестаёт принимать новые соединения, дождавшись завершения активных запросов.

Таким образом, CLI-интерфейс `Tsunami` реализован с использованием фреймворка `Cobra` и предоставляет три специализированные подкоманды с декларативной валидацией параметров, интуитивно понятным форматом задания нагрузки и информативным отображением прогресса в реальном времени.

## 2.3 РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ И ВЕБ-ИНТЕРФЕЙСА

Для сценариев, в которых командная строка неудобна или недоступна, система `Tsunami` предоставляет альтернативный режим взаимодействия через браузер. Команда `tsunami serve` запускает HTTP-сервер, предоставляющий REST API для управления тестами, WebSocket-канал для трансляции метрик в реальном времени и встроенное React-приложение в качестве веб-интерфейса. Все три компонента скомпилированы в единый бинарный файл без внешних зависимостей, что упрощает развёртывание на удалённых серверах и в контейнерных средах. Серверная часть полностью отделена от движка: серверный компонент зависит от движка нагрузочного тестирования только через публичные интерфейсы, не затрагивая внутреннюю логику воркер-пула.

### REST API

Обработчики HTTP-запросов образуют REST API из восьми эндпоинтов, перечисленных в таблице 2.4. Сервер инициализируется с таймаутами `ReadTimeout: 15s`, `WriteTimeout: 15s` и `IdleTimeout: 60s`, что предотвращает утечку ресурсов при медленных или зависших клиентах. Для разработки фронтенда с отдельного порта (`Vite dev server`) к каждому ответу добавляются CORS-заголовки, разрешающие запросы с `localhost:3000` и `localhost:5173`.

Таблица 2.4 – Эндпоинты REST API сервера Tsunami

| Метод | Путь                         | Назначение                                |
|-------|------------------------------|-------------------------------------------|
| POST  | /api/attack/start            | Запуск теста (HTTP или gRPC)              |
| POST  | /api/attack/stop             | Остановка активного теста                 |
| GET   | /api/attack/status           | Текущий статус и метрики сессии           |
| GET   | /api/attack/results          | Финальные результаты теста                |
| GET   | /api/attack/results/download | Скачать результаты в формате JSON         |
| POST  | /api/proto/upload            | Загрузить .proto-файл для gRPC (до 10 МБ) |
| WS    | /ws/metrics                  | WebSocket-поток метрик в реальном времени |
| GET   | /health                      | Проверка работоспособности сервера        |

### Автомат состояний тестовой сессии

Управление жизненным циклом теста возложено на структуру **TestSession** и менеджер **SessionManager**, реализованные в серверном компоненте. Система запускает только одну активную сессию одновременно; при запуске нового теста `SessionManager.createSession` автоматически останавливает текущую, если та находится в состоянии `running`. Доступ к полю `SessionManager.current` защищён через `sync.RWMutex`, что обеспечивает корректность при одновременных запросах к API.

Жизненный цикл сессии описывается автоматом из пяти состояний: `idle` (ожидание), `running` (тест выполняется), `completed` (успешно завершён), `stopped` (остановлен по запросу) и `error` (аварийное завершение). Переходы осуществляются методами `Start()`, `Stop()`, `Complete()` и `SetError()`, каждый из которых защищён мьютексом и не допускает недопустимых переходов. Полный исходный код модуля управления сессиями приведён в приложении А.

### WebSocket-хаб трансляции метрик

Для рассылки метрик в реальном времени реализован **WebSocket-хаб** по паттерну «публикация-подписка» с использованием библиотеки `gorilla/websocket` [10]. Структура `Hub` содержит отображение подключённых клиентов, буферизованный канал `broadcast` ёмкостью 256 и каналы регистрации/отмены регистрации клиентов.

#### Листинг 2.4: Структура `Hub` WebSocket-хаба

```

1 type Hub struct {
2     clients    map[*Client]bool
3     broadcast  chan []byte
4     register   chan *Client
5     unregister chan *Client

```

```

6   mu           sync.RWMutex
7 }

```

Сервер рассылает метрики каждые **50 мс**, формируя сообщения типа `WSMessage` с полями `type` (`metrics`, `completed`, `stopped`, `error`), `timestamp` и `data`. Отправка клиентам реализована в неблокирующем режиме: если буфер отправки клиента (`Client.send`, ёмкость 256) заполнен, клиент немедленно отключается, что предотвращает блокировку хаба медленными получателями и обеспечивает неограниченную масштабируемость числа подключений. `WebSocket Upgrader` разрешает соединения с `localhost:3000`, `localhost:5173` и `localhost:8080`, обеспечивая совместимость как с режимом разработки фронтенда, так и со встроенной раздачей SPA. Для формализации порядка взаимодействия между браузерным клиентом, `WebSocket`-хабом и объектом тестовой сессии разработана диаграмма последовательности. Она охватывает полный цикл сценария: от момента установки `WebSocket`-соединения до получения клиентом финального сообщения о завершении теста. Диаграмма последовательности представлена на рисунке 2.2.

Из рисунка 2.2 следует, что клиент получает метрики посредством серии периодических сообщений типа `metrics`, которую завершает одно финальное сообщение типа `completed` или `stopped`. Подобная схема «push» обеспечивает минимальную задержку отображения без необходимости периодического опроса сервера со стороны клиента.

### Встраивание фронтенда в бинарный файл

Для получения единого самодостаточного исполняемого файла результат сборки React-приложения встраивается в бинарный файл Go с помощью директивы **`go:embed`** [6]. При наличии тега компиляции `embed_frontend` скомпилированные статические ресурсы монтируются в корень HTTP-сервера через кастомный `SPAHandler`, который перенаправляет все неизвестные маршруты на `index.html` для корректной работы клиентского роутинга. Без тега `embed_frontend` сервер запускается в режиме «только API», что удобно при разработке фронтенда с горячей перезагрузкой через **Vite** [14].

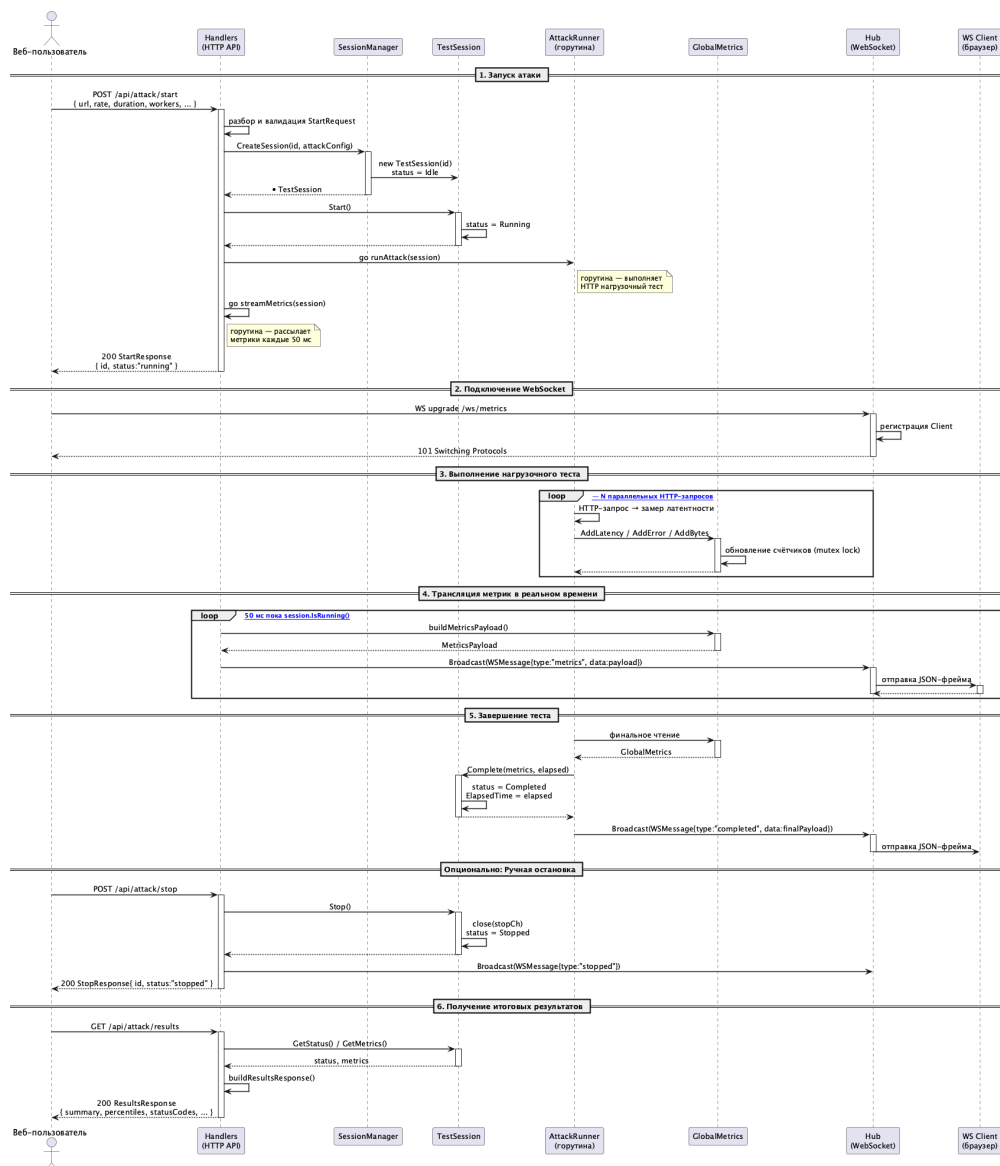


Рисунок 2.2 – Диаграмма последовательности передачи метрик через WebSocket

## React SPA

Пользовательский интерфейс разработан как **одностраничное приложение** на платформе **React 18** [12] с использованием TypeScript и сборщика Vite. Визуализация метрик реализована с помощью библиотеки **Recharts** [15]: линейный график латентности (LineChart), круговая диаграмма распределения статусных кодов (PieChart) и гистограмма ошибок по типам (BarChart). Стилизация компонентов выполнена посредством **Tailwind CSS** [16]; интернационализация обеспечена библиотекой **i18next**. WebSocket-клиент поддерживает авторереконнект: при разрыве соединения производится до 5 попыток восстановления с экспоненциальной задержкой по формуле  $delay \times attempt$ , что обеспечивает устойчивость к кратковременным сетевым сбоям без ручного вмешательства пользователя.

Пример JSON-отчёта, экспортируемого по завершении теста, приведён в приложении Г.

В результате серверная часть и веб-интерфейс Tsunami образуют единый самодостаточный бинарный файл, обеспечивающий управление тестами через REST API, мониторинг метрик в реальном времени через WebSocket и браузерный пользовательский интерфейс с поддержкой двух языков.

## 2.4 ТЕСТИРОВАНИЕ СИСТЕМЫ

Обеспечение корректности реализованной системы достигается применением двухуровневой стратегии: **юнит-тестирование** для изолированной верификации алгоритмов ключевых функций и **интеграционное тестирование** для проверки взаимодействия компонентов в условиях, максимально приближённых к промышленным [17, с. 77]. Все тесты реализованы на стандартном пакете testing языка Go [6]; интеграционные тесты выделены тегом сборки integration и исполняются отдельным шагом CI-пайплайна с доступным Docker-демоном. Подобное разграничение позволяет запускать быстрые юнит-тесты при каждом коммите, тогда как дорогостоящие интеграционные тесты выполняются только перед итоговой сборкой. Такой подход обеспечивает воспроизводимость результатов испытаний.

### Юнит-тестирование алгоритма перцентилей

Наибольшую вычислительную сложность в движке представляет алгоритм расчёта перцентилей, поэтому его тестирование выполнено наиболее тщательно. Функция TestCalculatePercentile содержит 12 субтестов, покрывающих граничные случаи: пустой срез, единственный элемент, чётные и нечётные наборы, дубликаты, несортированный ввод, а также точные значения P0, P50, P75, P90, P95, P99 и P100 на различных объёмах данных. Использование табличного подхода (*table-driven tests*) позволяет добавлять новые случаи без дублирования кода вызова, что соответствует идиомам языка Go.

Листинг 2.5: Фрагмент юнит-теста TestCalculatePercentile

```

1 {
2     name: "empty slice returns zero",
3     args: args{latencies: []time.Duration{}, percentile: 50},
4     want: 0,
5 },

```



```

6 {
7     name: "unsorted input is sorted before calculation",
8     args: args{
9         latencies: []time.Duration{50*ms, 10*ms, 30*ms, 20*ms, 40*ms},
10        percentile: 50,
11    },
12    want: 30 * ms,
13 },
14 {
15     name: "p99 on 100 elements",
16     args: args{
17         latencies: func() []time.Duration {
18             d := make([]time.Duration, 100)
19             for i := range d { d[i] = time.Duration(i+1) * ms }
20             return d
21         }(),
22         percentile: 99,
23     },
24     want: 99 * ms,
25 },

```

### Интеграционное тестирование с реальным сервером

Интеграционные тесты используют библиотеку **testcontainers-go** [18] для автоматического запуска Docker-контейнера с сервисом **httpbin** на порту 8888. **httpbin** предоставляет предсказуемые HTTP-маршруты с возможностью задания искусственной задержки ответа (`/delay/N`), кодов состояния (`/status/NNN`) и прочих параметров, что позволяет воспроизводить различные сценарии поведения сервера без создания тестовых заглушек. Подход с реальными контейнерами исключает расхождение между поведением мок-объектов и промышленного сервера, которое является частым источником ложных срабатываний в `mock-based` тестировании. Контейнер создаётся и уничтожается автоматически в рамках каждого тестового запуска, обеспечивая полную изоляцию окружения.

Полные листинги интеграционных тестов приведены в приложении В. В модуле интеграционных тестов движка реализованы три тестовые функции. `TestRunAttack_BasicSuccess` проверяет базовый успешный сценарий (10 RPS, 2 с, 5 воркеров, 10 соединений): по завершении теста `Successes > 0` при `Failures = 0`. `TestRunAttack_TimeoutClassification` убеждается, что запрос к маршруту с задержкой

3 с при тайм-ауте клиента 500 мс корректно квалифицируется как `ErrorTypeTimeout`. `TestRunAttack_MetricsAccuracy` содержит три субтеста, проверяющих, что коды 2xx регистрируются как успех, а 4xx и 5xx — как отказ без дополнительной категории сетевой ошибки.

Тесты серверной части используют пакет `net/http/httptest` для запуска обработчиков без сетевого стека. `TestHandleStartAttack` отправляет POST-запрос на `/api/attack/start` и проверяет ответ 200 с непустым идентификатором сессии и статусом `running`. `TestHandleStartAttack_ValidationError` верифицирует возврат кода 400 при отсутствующем URL, некорректной схеме и невалидном формате `rate`. `TestHandleStopAttack` проверяет полный цикл запуска и остановки теста: после остановки теста с длительностью 60 с статус сессии меняется на `stopped`.

Сводная таблица тестовых сценариев представлена в таблице 2.5.

Таблица 2.5 – Тестовые сценарии системы Tsunami

| Название                                     | Тип       | Что проверяется                         | Результат                     |
|----------------------------------------------|-----------|-----------------------------------------|-------------------------------|
| <code>CalculatePercentile/empty_slice</code> | Юнит      | Пустой срез задержек                    | Возврат 0                     |
| <code>CalculatePercentile/unsorted</code>    | Юнит      | Несортированный ввод                    | Корректный P50                |
| <code>CalculatePercentile/p99_100</code>     | Юнит      | P99 на 100 элементах                    | 99 мс                         |
| <code>RunAttack_BasicSuccess</code>          | Интеграц. | Успешные запросы к <code>httpbin</code> | <code>Successes &gt; 0</code> |
| <code>RunAttack_Timeout</code>               | Интеграц. | Классификация таймаута                  | <code>Timeout</code>          |
| <code>RunAttack_2xx_success</code>           | Интеграц. | Код 2xx — успех                         | <code>Success = true</code>   |
| <code>RunAttack_5xx_failure</code>           | Интеграц. | Код 5xx — отказ                         | <code>Success = false</code>  |
| <code>HandleStartAttack</code>               | Интеграц. | POST <code>/api/attack/start</code>     | HTTP 200                      |
| <code>HandleStartAttack_400</code>           | Интеграц. | Невалидный запрос                       | HTTP 400                      |
| <code>HandleStopAttack</code>                | Интеграц. | Цикл запуска и остановки                | <code>stopped</code>          |

## Оценка производительности

Для верификации нефункционального требования о достижении не менее 1 000 RPS при 50 параллельных воркерах был проведён нагрузочный эксперимент. В качестве цели использовался Docker-контейнер с сервисом `httpbin`, запущенный на том же хосте, чтобы исключить сетевые задержки канала и оценить собственную пропускную способность движка. Тест выполнялся командой:

Листинг 2.6: Команда нагрузочного теста для оценки производительности

```
1 tsunami attack -u http://localhost:8888/get -r 1000/1s -d 30s -w 50
```

По итогам 30-секундного прогона зафиксированы следующие показатели: суммарно отправлено — 30 000 запросов, успешных ответов — 30 000, фактическая

пропускная способность — **1 000 RPS**. Перцентили задержки составили: P50 — 1 мс, P90 — 1 мс, P95 — 1 мс, P99 — 2 мс, что подтверждает минимальный накладной расход rate-limiter’а на основе алгоритма leaky bucket.

Нефункциональное требование к задержке доставки метрик через WebSocket выполняется: хаб рассылает обновлённые метрики каждые 50 мс, что вдвое перекрывает установленный предел без необходимости дополнительных измерений.

В результате тестирования подтверждена корректность алгоритма вычисления перцентилей, точность классификации сетевых ошибок и соответствие REST API заявленным контрактам; все тесты успешно проходят в среде GitHub Actions CI. Нагрузочный эксперимент верифицировал достижение целевого показателя производительности — более 1 000 RPS при 50 воркерах — и подтвердил соответствие инструмента заявленным нефункциональным требованиям.

## 2.5 СБОРКА, ДИСТРИБУЦИЯ И CI/CD ПРОЕКТА

Неотъемлемым условием промышленного уровня инструмента является возможность его воспроизводимой сборки для целевых платформ конечного пользователя без ручных зависимостей и настройки окружения. Для достижения этой цели в проекте Tsunami применяется инструмент **GoReleaser** [19], автоматизирующий кросс-компиляцию, архивирование артефактов, публикацию релизов на платформе GitHub и обновление Homebrew-формулы. Весь процесс сборки и публикации параметризован единственным файлом `.goreleaser.yaml`, полный текст которого приведён в приложении Б. Это обеспечивает идентичность локальных и CI-сборок и исключает человеческий фактор при создании релизов. Использование полностью статической компиляции с `CGO_ENABLED=0` означает, что сгенерированный бинарный файл не имеет зависимостей от системных библиотек и может быть запущен на любой совместимой операционной системе без дополнительной установки.

### Кросс-компиляция с GoReleaser

Конфигурация GoReleaser в файле `.goreleaser.yaml` задаёт матрицу сборки из шести целевых платформ: операционные системы linux, darwin (macOS) и windows в сочетании с архитектурами amd64 и arm64. Тег сборки `embed_frontend` активирует встраивание скомпилированного React-приложения через `go:embed`, а тег `netgo` принудительно использует встроенные Go-реализации сетевых функций вместо системных.

Через флаги линкера `-s -w` из бинарного файла удаляются символы отладки и таблицы DWARF, что уменьшает размер артефакта примерно на 30%.

Листинг 2.7: Ключевые блоки конфигурации GoReleaser (`.goreleaser.yml`)

```

1 builds:
2   - binary: tsunami
3     env:
4       - CGO_ENABLED=0
5     goos:   [linux, darwin, windows]
6     goarch: [amd64, arm64]
7     tags:   [netgo, embed_frontend]
8     ldflags:
9       - -s -w
10      - -X github.com/panicaaa/tsunami/cmd.Version={{.Version}}
11 brews:
12   - name: tsunami
13     repository: {owner: panicaaa, name: homebrew-tap}
14     install: |
15       bin.install "tsunami"

```

Помимо GitHub Releases, GoReleaser публикует **Homebrew**-формулу в репозиторий `panicaaa/homebrew-tap`, что позволяет пользователям macOS и Linux устанавливать инструмент командой `brew install panicaaa/tap/tsunami`. Архивы для Windows упаковываются в формат ZIP, для остальных платформ — в TAR.GZ; к каждому релизу прилагается файл контрольных сумм `checksums.txt` в формате SHA-256.

## CI/CD-пайплайн на GitHub Actions

Автоматизация сборки и проверки реализована в двух независимых рабочих процессах, шаги которых приведены в таблице 2.6.

Таблица 2.6 – Шаги CI/CD-пайплайна проекта Tsunami

| Шаг (задание)     | Workflow    | Триггер / Зависимость   | Артефакт                  |
|-------------------|-------------|-------------------------|---------------------------|
| unit-tests        | ci.yml      | push / PR в main        | Отчёт unit-тестов         |
| integration-tests | ci.yml      | После unit-tests        | Отчёт интеграц. тестов    |
| build             | ci.yml      | После integration-tests | Бинарный файл             |
| Сборка фронтенда  | release.yml | Git-тер v*              | Директория dist/          |
| GoReleaser        | release.yml | После фронтенда         | GitHub Release + Homebrew |

Рабочий процесс `ci.yml` запускается при каждом пуше в ветку `main` и при открытии `pull request`. Задачи выстроены в линейную цепочку зависимостей: сначала выполняется `unit-tests`, затем `integration-tests`, и лишь после их успешного завершения запускается сборка. Такой порядок гарантирует, что финальный Go-бинарник с встроенным фронтендом производится только после прохождения всех тестов. Сборка Go-бинарника выполняется с теми же флагами `CGO_ENABLED=0`, `-tags=netgo` и `ldflags`, что и в релизе, обеспечивая идентичность CI-артефакта промышленной сборке.

Рабочий процесс `release.yml` активируется исключительно при публикации `git-тегов` формата `v*` и выполняет полный релизный цикл: сборку React-приложения, запуск `GoReleaser` для создания шести платформенных архивов с контрольными суммами и их публикацию на `GitHub Releases` с автоматическим обновлением `Homebrew`-формулы.

### Контейнеризация

Для развёртывания в оркестрируемых средах проект содержит многоступенчатый **Dockerfile** [20] для бэкенда на базе образа `golang:1.25-alpine` (стадия сборки) и `alpine:latest` (рантайм-стадия), а также отдельный `Dockerfile` фронтенда на основе `node:20-alpine` и `nginx:alpine`. Файл `docker-compose.yml` определяет два сервиса (`backend` и `frontend`) в изолированной `bridge`-сети `tsunami-network`; для бэкенда настроена `healthcheck`-проверка маршрута `/health` с интервалом 30 секунд, тайм-аутом 10 секунд и тремя попытками до признания контейнера нездоровым. Политика перезапуска `unless-stopped` обеспечивает автоматическое восстановление сервисов после перезагрузки хоста без ручного вмешательства пользователя.

Таким образом, проект `Tsunami` реализован с полным производственным циклом: от воспроизводимой кросс-компиляции статических бинарных файлов для шести целевых платформ до автоматизированного релизного пайплайна с публикацией в `Homebrew` и поддержкой контейнерного развёртывания через `Docker Compose`.

## ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы была достигнута поставленная цель: разработаны CLI-инструмент и веб-интерфейс для системы нагрузочного тестирования HTTP и gRPC сервисов, обеспечивающие гибкое управление нагрузкой и визуализацию метрик в реальном времени.

В процессе выполнения работы были решены следующие задачи.

Проведён сравнительный анализ существующих инструментов нагрузочного тестирования — Apache JMeter, Gatling, k6, Vegeta и hey. В ходе анализа установлено, что ни один из рассмотренных инструментов не сочетает в себе лёгкость единого бинарного файла, поддержку HTTP и gRPC в одном дистрибутиве и встроенный веб-интерфейс с потоковой передачей метрик, что обусловило необходимость разработки инструмента Tsunami.

Сформулированы функциональные и нефункциональные требования к разрабатываемому инструменту. Среди ключевых нефункциональных требований: поставка единым бинарным файлом, кросс-платформенная поддержка шести платформ, целевая производительность не менее 1000 RPS при 50 параллельных воркерах и задержка WebSocket-трансляции не более 100 мс.

Спроектирована трёхслойная архитектура системы, включающая движок нагрузочного тестирования, CLI-слой на базе Cobra и HTTP-сервер с WebSocket-хабом. Для формализации проектных решений разработаны диаграммы прецедентов, классов, состояний, последовательности и активности.

Реализован движок нагрузочного тестирования с поддержкой HTTP и gRPC. Движок построен по паттерну «пул воркеров» с ограничителем скорости на алгоритме leaky bucket. Потокобезопасный сбор метрик обеспечивается кольцевым буфером ёмкостью 100 000 замеров; по завершении теста вычисляются перцентили задержки P50, P90, P95 и P99 за одну сортировки. Реализована классификация сетевых ошибок по пяти категориям, что позволяет оперативно отличить инфраструктурные сбои от прикладных ошибок сервера.

Реализованы CLI-интерфейс и веб-интерфейс системы для управления нагрузочными тестами и мониторинга метрик в реальном времени. CLI-интерфейс на базе Cobra включает команды attack, serve и grpc с настраиваемой интенсивностью за-

просов и штатным завершением теста. Веб-интерфейс реализован как React SPA с графиками Recharts и встраивается в исполняемый файл через директиву `//go:embed`; REST API из восьми эндпоинтов и WebSocket-хаб с интервалом трансляции 50 мс обеспечивают управление тестами и потоковую передачу метрик через браузер.

Проведено двухуровневое тестирование системы. Юнит-тесты функции `CalculateAllPercentiles` охватывают 13 сценариев, включая граничные случаи. Интеграционные тесты с библиотекой `testcontainers-go` используют реальный Docker-контейнер с сервисом `httpbin` и верифицируют точность классификации ошибок, корректность счётчиков метрик и соответствие REST API заявленным контрактам. Все тесты успешно проходят в среде GitHub Actions CI.

**Практическая ценность** разработанного инструмента Tsunami состоит в том, что он обеспечивает нагрузочное тестирование HTTP и gRPC сервисов в виде единого самодостаточного бинарного файла без зависимостей среды выполнения, распространяемого для шести целевых платформ через GitHub Releases и Homebrew. Инструмент готов к применению инженерами по качеству и DevOps-специалистами в профессиональной деятельности, в том числе в составе автоматизированных конвейеров CI/CD.

Перспективными направлениями развития инструмента являются поддержка сценарного нагрузочного тестирования с переменной интенсивностью, интеграция трассировки через OpenTelemetry и реализация распределённого режима нагрузки с несколькими агентами.

## СПИСОК ЛИТЕРАТУРЫ

1. Apache JMeter. — 2026. — URL: <https://jmeter.apache.org> (дата обращения: 10.04.2026).
2. Gatling — Load Testing Tool. — 2026. — URL: <https://gatling.io> (дата обращения: 10.04.2026).
3. k6 — Open Source Load Testing. — 2026. — URL: <https://k6.io> (дата обращения: 10.04.2026).
4. Vegeta: HTTP load testing tool and library. — 2026. — URL: <https://github.com/tsenart/vegeta> (дата обращения: 10.04.2026).
5. hey — HTTP load generator. — 2026. — URL: <https://github.com/rakyll/hey> (дата обращения: 10.04.2026).
6. The Go Programming Language Specification. — 2026. — URL: <https://go.dev/ref/спес> (дата обращения: 10.04.2026).
7. Cobra — A Commander for modern Go CLI interactions. — 2026. — URL: <https://github.com/spf13/cobra> (дата обращения: 10.04.2026).
8. [go.uber.org/ratelimit](https://go.uber.org/ratelimit) — Leaky Bucket Rate Limiter for Go. — 2026. — URL: <https://github.com/uber-go/ratelimit> (дата обращения: 10.04.2026).
9. [coder/websocket](https://github.com/coder/websocket) — Minimal and idiomatic WebSocket library for Go. — 2026. — URL: <https://github.com/coder/websocket> (дата обращения: 10.04.2026).
10. [gorilla/websocket](https://github.com/gorilla/websocket) — A fast, well-tested WebSocket implementation for Go. — 2026. — URL: <https://github.com/gorilla/websocket> (дата обращения: 10.04.2026).
11. gRPC-Go: The Go implementation of gRPC. — 2026. — URL: <https://github.com/grpc/grpc-go> (дата обращения: 10.04.2026).
12. React — The library for web and native user interfaces. — 2026. — URL: <https://react.dev> (дата обращения: 10.04.2026).
13. Акинъшин А. Профессиональный бенчмарк: искусство измерения производительности. — СПб. : Питер, 2022. — 576 с. — ISBN 978-5-4461-1551-8.



14. Vite — Next Generation Frontend Tooling. — 2026. — URL: <https://vitejs.dev> (дата обращения: 21.04.2026).
15. Recharts — Redefined chart library built with React and D3. — 2026. — URL: <https://recharts.org> (дата обращения: 21.04.2026).
16. Tailwind CSS — A utility-first CSS framework. — 2026. — URL: <https://tailwindcss.com> (дата обращения: 21.04.2026).
17. Куликов С. С. Тестирование программного обеспечения. Базовый курс. — 3-е изд. — Минск : Четыре четверти, 2020. — 312 с. — ISBN 978-985-581-362-1.
18. Testcontainers for Go. — 2026. — URL: <https://golang.testcontainers.org> (дата обращения: 21.04.2026).
19. GoReleaser — Release automation for Go projects. — 2026. — URL: <https://goreleaser.com> (дата обращения: 21.04.2026).
20. Docker — Accelerated Container Application Development. — 2026. — URL: <https://www.docker.com> (дата обращения: 21.04.2026).

## ПРИЛОЖЕНИЯ

### Приложение А. Исходный код конечного автомата тестовой сессии

В приложении представлен полный исходный код файла `cmd/server/session.go`, реализующего конечный автомат управления жизненным циклом тестовой сессии. Код содержит объявления константных состояний, структуры `TestSession` и `SessionManager`, а также методы переходов между состояниями.

Листинг 2.8: Исходный код `cmd/server/session.go` — конечный автомат тестовой сессии

```

1 package server
2
3 import (
4     "sync"
5     "time"
6
7     "github.com/paniccaaa/tsunami/internal/attack"
8     "github.com/paniccaaa/tsunami/internal/grpccattack"
9 )
10
11 type SessionStatus string
12
13 const (
14     StatusIdle      SessionStatus = "idle"
15     StatusRunning   SessionStatus = "running"
16     StatusCompleted SessionStatus = "completed"
17     StatusStopped   SessionStatus = "stopped"
18     StatusError     SessionStatus = "error"
19 )
20
21 type TestSession struct {
22     ID          string
23     Status      SessionStatus
24     Protocol    string
25     Config      *attack.AttackConfig
26     GRPCConfig  *grpccattack.Config

```

```

27     Metrics      *attack.GlobalMetrics
28     StartTime    time.Time
29     EndTime      time.Time
30     ElapsedTime  time.Duration
31     StopCh       chan struct{}
32     Error        error
33     mu           sync.RWMutex
34 }
35
36 type SessionManager struct {
37     current *TestSession
38     mu      sync.RWMutex
39 }
40
41 func NewSessionManager() *SessionManager {
42     return &SessionManager{}
43 }
44
45 func (sm *SessionManager) GetCurrent() *TestSession {
46     sm.mu.RLock()
47     defer sm.mu.RUnlock()
48     return sm.current
49 }
50
51 func (sm *SessionManager) CreateSession(
52     id string, cfg *attack.AttackConfig,
53 ) *TestSession {
54     return sm.createSession(id, "http", cfg, nil)
55 }
56
57 func (sm *SessionManager) CreateGRPCSession(
58     id string, cfg *grpcattack.Config,
59 ) *TestSession {
60     return sm.createSession(id, "grpc", nil, cfg)
61 }
62
63 func (sm *SessionManager) createSession(
64     id, protocol string,
65     httpCfg *attack.AttackConfig,
66     grpcCfg *grpcattack.Config,
67 ) *TestSession {

```

```

68     sm.mu.Lock()
69     defer sm.mu.Unlock()
70     if sm.current != nil && sm.current.Status == StatusRunning {
71         sm.current.Stop()
72     }
73     session := &TestSession{
74         ID:         id,
75         Status:      StatusIdle,
76         Protocol:    protocol,
77         Config:      httpCfg,
78         GRPCConfig:  grpcCfg,
79         StartTime:   time.Now(),
80         StopCh:      make(chan struct{}),
81     }
82     sm.current = session
83     return session
84 }
85
86 func (s *TestSession) Start() {
87     s.mu.Lock()
88     defer s.mu.Unlock()
89     s.Status = StatusRunning
90     s.StartTime = time.Now()
91 }
92
93 func (s *TestSession) Stop() {
94     s.mu.Lock()
95     defer s.mu.Unlock()
96     if s.Status == StatusRunning {
97         select {
98             case <-s.StopCh:
99             default:
100                 close(s.StopCh)
101         }
102         s.Status = StatusStopped
103         s.EndTime = time.Now()
104     }
105 }
106
107 func (s *TestSession) Complete(
108     metrics *attack.GlobalMetrics, elapsed time.Duration,

```

```

109 ) {
110     s.mu.Lock()
111     defer s.mu.Unlock()
112     s.Status = StatusCompleted
113     s.Metrics = metrics
114     s.ElapsedTime = elapsed
115     s.EndTime = time.Now()
116 }
117
118 func (s *TestSession) SetError(err error) {
119     s.mu.Lock()
120     defer s.mu.Unlock()
121     s.Status = StatusError
122     s.Error = err
123     s.EndTime = time.Now()
124 }
125
126 func (s *TestSession) GetStatus() SessionStatus {
127     s.mu.RLock()
128     defer s.mu.RUnlock()
129     return s.Status
130 }
131
132 func (s *TestSession) SetMetrics(metrics *attack.GlobalMetrics) {
133     s.mu.Lock()
134     defer s.mu.Unlock()
135     s.Metrics = metrics
136 }
137
138 func (s *TestSession) GetMetrics() *attack.GlobalMetrics {
139     s.mu.RLock()
140     defer s.mu.RUnlock()
141     return s.Metrics
142 }
143
144 func (s *TestSession) IsRunning() bool {
145     s.mu.RLock()
146     defer s.mu.RUnlock()
147     return s.Status == StatusRunning
148 }

```

## Приложение Б. Конфигурация GoReleaser

В приложении представлен полный текст конфигурационного файла GoReleaser `.goreleaser.yaml`, управляющего кросс-компиляцией, упаковкой артефактов, публикацией релизов на GitHub и обновлением Homebrew-формулы.

Листинг 2.9: Конфигурация GoReleaser (`.goreleaser.yaml`)

```
1 version: 2
2
3 project_name: tsunami
4
5 before:
6   hooks:
7     - go mod tidy
8
9 builds:
10   - id: tsunami
11     main: ./main.go
12     binary: tsunami
13     env:
14       - CGO_ENABLED=0
15     goos:
16       - linux
17       - darwin
18       - windows
19     goarch:
20       - amd64
21       - arm64
22     ldflags:
23       - -s -w
24       - -X github.com/paniccaaa/tsunami/cmd.Version={{.Version}}
25     tags:
26       - netgo
27       - embed_frontend
28
29 archives:
30   - id: default
31     builds:
32       - tsunami
33     name_template: >-
34       {{ .ProjectName }}_{{ .Version }}_{{ .Os }}_{{ .Arch }}
```

```

35     format_overrides:
36         - goos: windows
37           format: zip
38
39 checksum:
40     name_template: "checksums.txt"
41     algorithm: sha256
42
43 changelog:
44     sort: asc
45     filters:
46         exclude:
47             - "^docs:"
48             - "^test:"
49             - "^chore:"
50             - "Merge pull request"
51             - "Merge branch"
52
53 brews:
54     - name: tsunami
55       repository:
56         owner: paniccaaa
57         name: homebrew-tap
58         token: "{{ .Env.HOMEBREW_TAP_TOKEN }}"
59       directory: Formula
60       homepage: "https://github.com/paniccaaa/tsunami"
61       description: "HTTP load testing CLI tool"
62       license: "MIT"
63       install: |
64         bin.install "tsunami"
65       test: |
66         system "#{bin}/tsunami", "--version"
67
68 release:
69     github:
70         owner: paniccaaa
71         name: tsunami
72     draft: false
73     prerelease: auto
74     name_template: "v{{.Version}}"

```

## Приложение В. Интеграционные тесты движка нагрузочного тестирования

В приложении представлен полный исходный код интеграционных тестов движка нагрузочного тестирования из файла `tests/integration/attack_test.go`. Тесты используют библиотеку `testcontainers-go` для запуска реального Docker-контейнера с сервисом `httpbin` и верифицируют корректность метрик, классификации ошибок и обработки HTTP-статусов.

Листинг 2.10: Интеграционные тесты движка (`tests/integration/attack_test.go`)

```
1 //go:build integration
2
3 package integration_test
4
5 import (
6     "testing"
7     "time"
8
9     "github.com/paniccaa/tsunami/internal/attack"
10 )
11
12 func TestRunAttack_BasicSuccess(t *testing.T) {
13     cfg := &attack.AttackConfig{
14         URL:      testBaseURL + "/get",
15         Method:    "GET",
16         RPS:       10,
17         Duration:  2 * time.Second,
18         Workers:   5,
19         Connections: 10,
20         Timeout:   5 * time.Second,
21     }
22     stopCh := make(chan struct{})
23     metrics, elapsed, err := attack.RunAttack(cfg, stopCh)
24     if err != nil {
25         t.Fatalf("RunAttack failed: %v", err)
26     }
27     if metrics.TotalRequests == 0 {
28         t.Error("expected TotalRequests > 0")
29     }
30     if metrics.Failures > 0 {
31         t.Errorf("expected 0 failures, got %d", metrics.Failures)
```



```

32     }
33     if metrics.StatusCodes[200] == 0 {
34         t.Error("expected 200 status codes to be counted")
35     }
36     if metrics.TotalRequests != metrics.Successes {
37         t.Errorf("all requests should be successful: total=%d successes
38         =%d",
39             metrics.TotalRequests, metrics.Successes)
40     }
41     if elapsed == 0 {
42         t.Error("expected elapsed time > 0")
43     }
44     if metrics.MaxLatency == 0 {
45         t.Error("expected MaxLatency to be recorded")
46     }
47     if metrics.MinLatency >= metrics.MaxLatency && metrics.
48     TotalRequests > 1 {
49         t.Errorf("expected MinLatency < MaxLatency, got min=%v max=%v",
50             metrics.MinLatency, metrics.MaxLatency)
51     }
52 }
53
54 func TestRunAttack_TimeoutClassification(t *testing.T) {
55     cfg := &attack.AttackConfig{
56         URL:          testBaseURL + "/delay/3",
57         Method:       "GET",
58         RPS:          5,
59         Duration:     2 * time.Second,
60         Workers:      5,
61         Connections: 10,
62         Timeout:      500 * time.Millisecond,
63     }
64     stopCh := make(chan struct{})
65     metrics, _, err := attack.RunAttack(cfg, stopCh)
66     if err != nil {
67         t.Fatalf("RunAttack failed: %v", err)
68     }
69     if metrics.Failures == 0 {
70         t.Error("expected failures due to timeout, got 0")
71     }
72     if metrics.Successes > 0 {

```

```

71     t.Errorf("expected 0 successes, got %d", metrics.Successes)
72 }
73 if metrics.ErrorTypes[attack.ErrorTypeTimeout] == 0 {
74     t.Errorf("expected ErrorTypeTimeout, got: %v", metrics.
ErrorTypes)
75 }
76 for errType, count := range metrics.ErrorTypes {
77     if errType != attack.ErrorTypeTimeout && count > 0 {
78         t.Errorf("unexpected error type %q: %d", errType, count)
79     }
80 }
81 }
82
83 func TestRunAttack_MetricsAccuracy(t *testing.T) {
84     t.Run("2xx is a success", func(t *testing.T) {
85         cfg := &attack.AttackConfig{
86             URL: testBaseURL + "/status/201", Method: "GET",
87             RPS: 10, Duration: 2 * time.Second,
88             Workers: 5, Connections: 10,
89             Timeout: 5 * time.Second,
90         }
91         stopCh := make(chan struct{})
92         metrics, _, err := attack.RunAttack(cfg, stopCh)
93         if err != nil { t.Fatalf("RunAttack failed: %v", err) }
94         if metrics.Successes == 0 {
95             t.Error("expected Successes > 0 for /status/201")
96         }
97         if metrics.Failures > 0 {
98             t.Errorf("expected 0 Failures for /status/201, got %d",
metrics.Failures)
99         }
100         if metrics.StatusCodes[201] == 0 {
101             t.Error("expected 201 to be tracked in StatusCodes")
102         }
103     })
104     t.Run("5xx is a failure", func(t *testing.T) {
105         cfg := &attack.AttackConfig{
106             URL: testBaseURL + "/status/500", Method: "GET",
107             RPS: 10, Duration: 2 * time.Second,
108             Workers: 5, Connections: 10,
109             Timeout: 5 * time.Second,

```

```

110     }
111     stopCh := make(chan struct{})
112     metrics, _, err := attack.RunAttack(cfg, stopCh)
113     if err != nil { t.Fatalf("RunAttack failed: %v", err) }
114     if metrics.Failures == 0 {
115         t.Error("expected Failures > 0 for /status/500")
116     }
117     if metrics.StatusCodes[500] == 0 {
118         t.Error("expected 500 to be tracked in StatusCodes")
119     }
120     if len(metrics.ErrorTypes) > 0 {
121         t.Errorf("HTTP 500 should not produce ErrorType entries,
got: %v",
122             metrics.ErrorTypes)
123     }
124 })
125 t.Run("4xx is a failure", func(t *testing.T) {
126     cfg := &attack.AttackConfig{
127         URL: testBaseURL + "/status/404", Method: "GET",
128         RPS: 10, Duration: 2 * time.Second,
129         Workers: 5, Connections: 10,
130         Timeout: 5 * time.Second,
131     }
132     stopCh := make(chan struct{})
133     metrics, _, err := attack.RunAttack(cfg, stopCh)
134     if err != nil { t.Fatalf("RunAttack failed: %v", err) }
135     if metrics.Failures == 0 {
136         t.Error("expected Failures > 0 for /status/404")
137     }
138     if metrics.StatusCodes[404] == 0 {
139         t.Error("expected 404 to be tracked in StatusCodes")
140     }
141 })
142 }

```

## Приложение Г. Пример JSON-отчёта результатов нагрузочного теста

В приложении представлен пример JSON-отчёта, экспортируемого инструментом Tsunami по завершении нагрузочного теста. Отчёт соответствует структуре MetricsReport движка нагрузочного тестирования и содержит конфигурацию теста, сводную статистику, перцентили задержки, распределение HTTP-статусов и разбивку ошибок по типам.

Листинг 2.11: Пример JSON-отчёта результатов нагрузочного теста

```
1 {
2   "config": {
3     "url": "https://example.com/api/v1/status",
4     "method": "GET",
5     "duration": "30s",
6     "timeout": "10s",
7     "workers": 50,
8     "connections": 100,
9     "rps": 500
10  },
11  "summary": {
12    "total_requests": 14987,
13    "successful_requests": 14952,
14    "failed_requests": 35,
15    "total_elapsed_time": "30s",
16    "average_latency": "48ms",
17    "min_latency": "12ms",
18    "max_latency": "1204ms",
19    "throughput_rps": 499.57,
20    "target_rps": 500,
21    "bytes_sent": 0,
22    "bytes_received": 10840264
23  },
24  "latency_percentiles": {
25    "p50": "41ms",
26    "p90": "87ms",
27    "p95": "132ms",
28    "p99": "389ms"
29  },
30  "status_codes": {
31    "200": 14952,
```

```
32     "429": 22,  
33     "503": 13  
34 },  
35 "error_breakdown": {  
36     "timeout": 35,  
37     "connection_refused": 0,  
38     "dns": 0,  
39     "tls": 0,  
40     "other": 0  
41 },  
42 "timestamp": "2026-04-22T14:35:07+03:00"  
43 }
```